ImpactLedger For NGO Funding Analysis And Tracking

Project Report submitted to

Ramdeobaba University, Nagpur

in partial fulfillment of requirement for the award of degree of

Bachelor of Technology (B.Tech)

In

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Cyber Security)

By

Saptanshu Wanjari (41)
Kamal Mann (62)
Pranjal Chaubey (37)
Gourang Daware (27)
Nikhil Singh (55)
Bharat Diwan (61)

IV Semester

Guide
Prof. ABHINAY GUDADHE

Department of Artificial Intelligence and Cyber Security
Ramdeobaba University, Nagpur 440 013
May 2026
RAMDEOBABA UNIVERSITY, NAGPUR
Department of Artificial intelligence and Cyber Security

1 CERTIFICATE

This is to certify that the Community Engagement Project titled "ImpactLedger For NGO Funding Analysis" is a Bonafide work of Saptanshu Wanjari, Kamal Mann, Pranjal Chaubey, Gourang Daware, Nikhil Singh, Bharat Diwan, submitted to the Ramdeobaba University, Nagpur in partial fulfilment of the award of a Degree of Bachelor of Technology (B.Tech), in Computer Science and Engineering (Artificial Intelligence and Cyber Security). The project was conducted in collaboration with Lions Club Nagpur Sawaj. It has been carried out at the Department of Artificial Intelligence and Cyber Security, Ramdeobaba University, Nagpur during the academic year 2025-2026.

Date:
Place: Nagpur

SANDIP BUTLE
FOUNDER
LIONS CLUB NAGPUR SAWAJ

Prof. ABHINAY GUDADHE
PROJECT GUIDE
DEPARTMENT OF AICS

DR. RASHMI WELEKAR
H.O.D
DEPARTMENT OF AICS

2 DECLARATION

We hereby declare that the project report titled "ImpactLedger For NGO Funding Analysis" submitted herein, has been carried out in the Department of Artificial Intelligence and Cyber Security of Ramdeobaba University, Nagpur. The work is original and has not been submitted earlier as a whole or part for the award of any degree/diploma at this or any other institution / University.

Date:
Place: Nagpur

Name, roll no and sign of all students
Saptanshu Wanjari (41)
Kamal Mann (62)
Pranjal Chaubey (37)
Gourang Daware (27)
Nikhil Singh (55)
Bharat Diwan (61)

3 ACKNOWLEDGEMENTS

We would like to sincerely express our gratitude to everyone who contributed to the successful completion of our Community Engagement Project (CEP) titled ImpactLedger For NGO Funding Analysis.

First and foremost, we are deeply thankful to our project guide, Prof. Abhinay Gudadhe, Department of Artificial Intelligence and Cyber Security, Ramdeobaba University, Nagpur, for his continuous guidance, valuable suggestions, and constant support throughout the development of this project. His mentorship played a crucial role in shaping our approach and improving the overall quality of our work.

We would also like to extend our sincere thanks to Ms. Rashmi Welekar, Head of the Department of Artificial Intelligence and Cyber Security, for providing the necessary academic environment and encouragement that enabled us to successfully carry out this project.

We are extremely grateful to Mr. Sandip Butle, Founder of Lions Club Nagpur Sawaj, for giving us the opportunity to work on a meaningful community-based initiative. His support and vision inspired us to develop a solution that promotes transparency and accountability in social initiatives.

We also express our heartfelt thanks to all the faculty members of the Department of Artificial Intelligence and Cyber Security who supported and guided us, directly or indirectly, throughout the project.

Finally, we would like to thank our parents and family members for their constant encouragement, motivation, and support, which helped us complete this project successfully.

4 ABSTRACT

ImpactLedger For NGO Funding Analysis is a web-based platform developed to enhance transparency, accountability, and efficiency in community-driven initiatives. Many non-governmental organizations (NGOs) and social projects currently rely on manual or unstructured systems for managing donations, tracking activities, and measuring impact. This often leads to issues such as lack of visibility, inefficient reporting, and reduced trust among stakeholders.

The proposed system addresses these challenges by providing a centralized platform where NGOs, donors, and volunteers can interact seamlessly. ImpactLedger enables users to create and manage campaigns, record field activity through volunteer modules, track donations, and monitor overall impact through a structured and organized system. The platform also offers data visualization features such as dashboards, charts, and progress indicators.

Built using modern web technologies such as React/Next.js for the frontend, Next.js route handlers for backend services, and Supabase PostgreSQL for data storage, the system supports secure role-based access and auditable workflows. Razorpay integration and webhook verification provide payment traceability, while ledger tables and reporting views improve transparency.

By providing near real-time visibility into project progress and resource utilization, ImpactLedger supports better decision-making and accountability in community engagement projects.

Contents

1 CERTIFICATE **1**

2 DECLARATION **2**

3 ACKNOWLEDGEMENTS **3**

4 ABSTRACT **4**

5 LIST OF FIGURES **7**

6 LIST OF TABLES **7**

7 CHAPTER 1 INTRODUCTION **8**

8 CHAPTER 2 COMMUNITY ANALYSIS **9**

9 CHAPTER 3 LITERATURE REVIEW **10**

10 CHAPTER 4 SYSTEM REQUIREMENTS **11**

10.1 Table 4.1: Functional Requirements of ImpactLedger 11

10.2 Table 4.2: Non-Functional Requirements of ImpactLedger 12

10.3 Table 4.3: Hardware and Software Requirements 12

11 CHAPTER 5 SYSTEM DESIGN **13**

11.1 Figure 5.5: UI Wireframe Notes 15

12 CHAPTER 6 METHODOLOGY **15**

13 CHAPTER 7 IMPLEMENTATION **17**

13.1 Table 7.1: Screenshot Placement Guide for Chapter 7 18

14 CHAPTER 8

RESULTS AND IMPACT **19**

14.1 Table 8.1: Before vs After Comparison — ImpactLedger Implementation 19

14.2 Table 8.2: System Performance Metrics — ImpactLedger 20

15 CHAPTER 9 DISCUSSION **20**

16 CHAPTER 10 CONCLUSION AND FUTURE SCOPE **20**

17 QUICK REFERENCE: ALL FIGURES AND TABLES **21**

18 REFERENCES **21**

19 APPENDICES **21**

5 LIST OF FIGURES

- Figure 5.1: System Architecture Diagram
- Figure 5.2: Entity-Relationship Diagram
- Figure 5.3: Route and Module Interaction View
- Figure 5.4: Sequence Diagram — Donation Processing Flow
- Figure 6.1: Development Methodology Workflow
- Figure 6.2: Technology Stack Overview

6 LIST OF TABLES

- Table 4.1: Functional Requirements of ImpactLedger
- Table 4.2: Non-Functional Requirements of ImpactLedger
- Table 4.3: Hardware and Software Requirements
- Table 7.1: Screenshot Placement Guide for Chapter 7
- Table 8.1: Before vs After Comparison — ImpactLedger Implementation
- Table 8.2: System Performance Metrics — ImpactLedger

7 CHAPTER 1 INTRODUCTION

Community Engagement Projects (CEPs) serve as a bridge between academic learning and real-world problem-solving, enabling students to design and deploy technology-driven solutions for societal challenges. These projects provide an opportunity to apply theoretical knowledge in practical settings while contributing meaningfully to communities that often lack access to modern digital infrastructure. ImpactLedger is one such initiative, developed in collaboration with Lions Club Nagpur Sawaj to address a persistent gap in the operational transparency of non-governmental organizations (NGOs).

In the current landscape, NGOs and community-driven projects rely heavily on manual or informal methods for tracking activities, recording donations, and reporting outcomes. This approach not only introduces the risk of human error and data inconsistency, but also reduces the confidence of donors and other stakeholders in the overall management of projects. The absence of structured digital systems leads to delayed reporting, unverified claims of impact, and an overall decline in accountability.

Digital solutions play a crucial role in modern community engagement projects by enabling structured data management, near real-time monitoring, and improved transparency. Unlike traditional manual systems, digital platforms ensure accurate record-keeping and reduce the risk of data loss or inconsistency. They also allow stakeholders to access project information instantly, regardless of location. This improves accountability, strengthens donor confidence, and enables better decision-making through data-driven insights.

The primary objective of ImpactLedger is to develop a centralized, web-based platform that enables NGOs to record and manage their activities, track incoming contributions, and present verified impact data through interactive dashboards. By digitizing these processes, the platform ensures that all stakeholders, including donors, volunteers, and administrators, have access to accurate and up-to-date information.

The core problem addressed by this project is the lack of structured transparency mechanisms in community initiatives, which results in mismanagement of resources, weakened donor relationships, and inefficient impact reporting. ImpactLedger aims to resolve this by providing a reliable digital ecosystem for NGO operations.

The scope of the project covers the complete design and development of a web application with features for user authentication, campaign and assignment management, donation recording, and data visualization. The system is intended for use by NGOs registered on the platform and their associated stakeholders.

8 CHAPTER 2 COMMUNITY ANALYSIS

The target community for ImpactLedger comprises four distinct groups: NGO administrators, field volunteers, donors, and beneficiaries. NGO administrators are responsible for planning and executing social initiatives, volunteers assist in ground-level implementation, donors provide financial and material support, and beneficiaries are the individuals or groups who directly receive services. Understanding the specific roles and interactions of these groups was essential for designing a platform that genuinely addresses their needs.

Interactions with Lions Club Nagpur Sawaj, along with informal surveys and discussions with NGO coordinators and volunteers, revealed several operational challenges. Most organizations maintain records using physical ledgers or fragmented spreadsheets, which are prone to inconsistencies and difficult to audit. There is no single system through which all stakeholders can view project progress or verify the utilization of donated resources. This lack of a unified platform results in delayed reporting, reduced accountability, and declining donor participation.

A web-based solution is well-suited to address these challenges. A digital platform can automate routine record-keeping, provide centralized access to project data, and offer timely visibility into organizational activities. Unlike offline methods, a web application is accessible from any device with an internet connection, making it convenient for both urban and semi-urban users. It also enables systematic storage and retrieval of historical data, which is valuable for long-term impact assessment.

The key stakeholders whose requirements shaped the design of ImpactLedger are NGO administrators, who need efficient tools for campaign operations and reporting; donors, who require transparent financial tracking and impact visibility; and volunteers, who need assignment and field reporting surfaces. The system has been designed to serve each of these groups through a unified yet role-aware interface.

9 CHAPTER 3 LITERATURE REVIEW

Several web-based platforms currently exist to support NGO operations and charitable fundraising. Prominent examples include GiveIndia and GlobalGiving, which primarily focus on crowdfunding and project listing. These platforms allow organizations to create fundraising campaigns and receive donations, but they offer limited support for detailed operational tracking, resource utilization reporting, or outcome-based auditability. Their design often prioritizes fundraising over operational accountability.

In terms of technology, modern NGO management portals commonly use JavaScript-based frontend frameworks and API-backed server runtimes. These stacks are capable of building scalable and responsive applications, but existing platforms have not always leveraged them for comprehensive stewardship pipelines.

A critical limitation of many current solutions is the absence of integrated reporting with auditable event trails. Some platforms display data in static formats that are not easily interpretable by non-technical users. Furthermore, few platforms allow NGOs to maintain detailed volunteer activity logs and reconcile donation events with transparent ledger-style records.

ImpactLedger is designed to address these limitations by integrating dashboard-based visibility, role-aware workflows, payment reconciliation, and structured transparency outputs in a single platform. Unlike single-purpose fundraising-only systems, ImpactLedger supports the lifecycle from campaign publication and donation capture to verification and reporting.

10 CHAPTER 4 SYSTEM REQUIREMENTS

The functional and non-functional requirements are retained in chapter format, but updated to match implemented behavior.

10.1 Table 4.1: Functional Requirements of ImpactLedger

Req ID	Feature	Description	Priority
FR-01	User Registration & Login	Support signup/login and role-based landing paths for donor, volunteer, and org admin.	High
FR-02	Role-Based Access Control	Restrict API and page access according to tenant membership role.	High
FR-03	Campaign Management	Create, view, and manage campaigns via admin routes.	High
FR-04	Volunteer Activity Logging	Record assignment-linked volunteer reports and logs.	High
FR-05	Donation Recording	Create pending donations, reconcile payment events, and persist ledger history.	High
FR-06	Impact Dashboard	Display campaign, donor, and operations metrics via dashboard views.	High
FR-07	Reporting	Provide report and analytics endpoints for administrative review.	Medium
FR-08	Transparency View	Expose public transparency ledger and overview endpoints.	Medium
FR-09	Search and Filter	Filter donations and list views where implemented in route handlers/pages.	Medium
FR-10	Session Management	Handle auth sessions through middleware and Supabase auth context checks.	High

10.2 Table 4.2: Non-Functional Requirements of ImpactLedger

Req ID	Category	Requirement Description
NFR-01	Performance	Build and API test flows should complete under standard deployment conditions; route handlers are designed for operational dashboard usage.
NFR-02	Security	Use signed webhook verification, role checks, and controlled server-side data access.
NFR-03	Usability	Provide responsive web pages for public and role-specific workflows.
NFR-04	Maintainability	Keep modular server helpers for auth, data, payments, invoice, and PDF generation.
NFR-05	Reliability	Persist webhook and ledger events for traceability and replay-safe handling.
NFR-06	Responsiveness	Pages render on desktop/mobile breakpoints in current Next.js UI.
NFR-07	Data Privacy	Restrict role-protected data endpoints and avoid exposing private records publicly.
NFR-08	Observability	Maintain audit logs, donation ledger, and payment webhook event history.

10.3 Table 4.3: Hardware and Software Requirements

Environment	Component	Specification
Client Side	Hardware	Any modern device with browser and internet connectivity.
Client Side	Browser	Current Chrome/Firefox/Edge/Safari class browsers.
Server Side	Runtime	Node.js runtime used by Next.js deployment.
Server Side	Database	Supabase PostgreSQL (schema and migrations in repository).
Server Side	Payment Integration	Razorpay keys and webhook secret configured in environment.
Server Side	Email	SMTP credentials for donation invoice dispatch.
Development	IDE/Tooling	VS Code, Git/GitHub, npm scripts, and Vitest test suite.
Development	Validation	`npm run build`, `npm run test`, and LaTeX compile workflow.

11 CHAPTER 5 SYSTEM DESIGN

ImpactLedger follows a layered web architecture. The presentation layer consists of a Next.js/React frontend application that handles user interactions and renders dashboards. The application layer consists of Next.js route handlers and server modules that process requests, enforce role constraints, and coordinate payment and reporting flows. The data layer uses Supabase PostgreSQL for persistent storage of tenant records, campaigns, donors, donations, volunteer operations, ledgers, and audit logs. Communication between frontend and backend surfaces is handled through HTTP APIs.

The data model is structured around tenant-scoped relational entities rather than the previously claimed five-entity document model. Core tables include tenants, memberships, campaigns, donations, donors, volunteer profiles and assignments, expenses, donation/allocation ledgers, audit logs, and webhook events. This model improves traceability and supports reporting views such as campaign funding summary and transparency ledger.

The system design includes architecture and sequence diagrams to model behavior across core flows such as checkout-session creation, webhook reconciliation, and invoice dispatch.

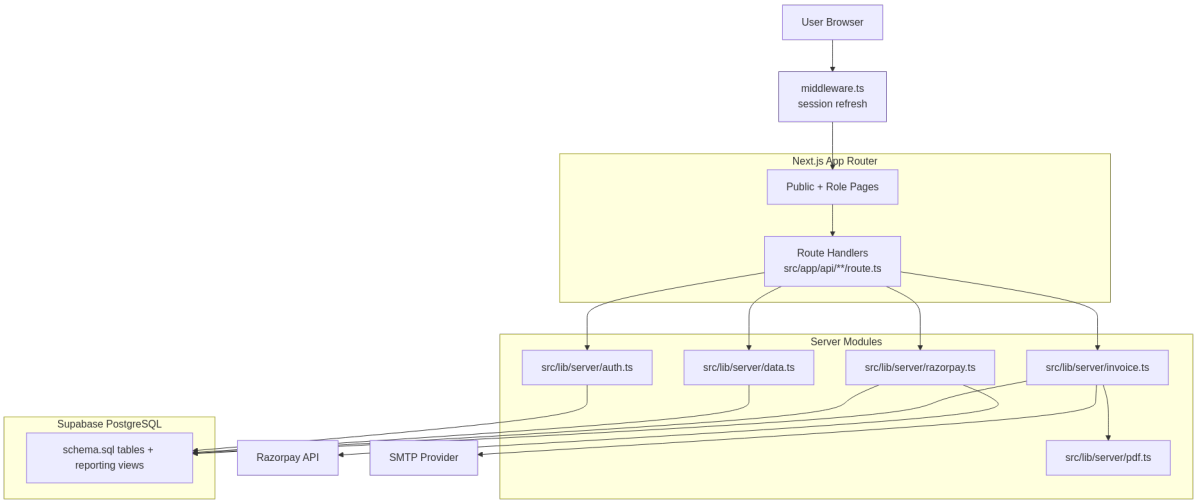

Figure 1: Figure 5.1: System Architecture Diagram (implemented components).

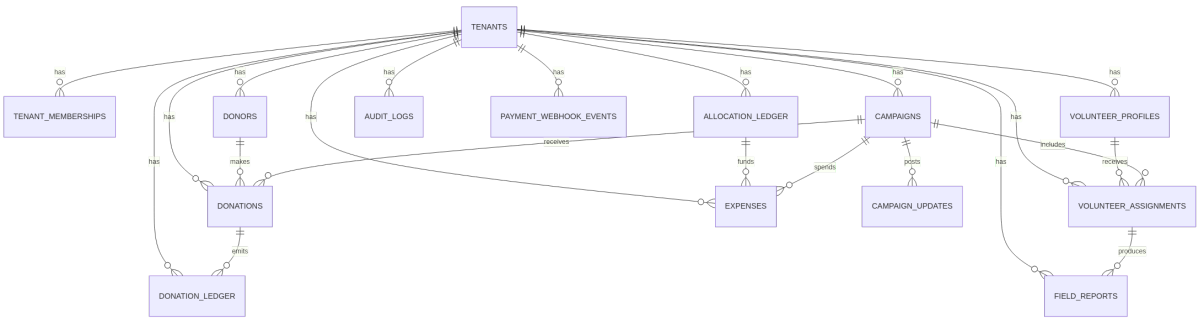

Figure 2: Figure 5.2: Entity-Relationship Diagram based on Supabase schema.

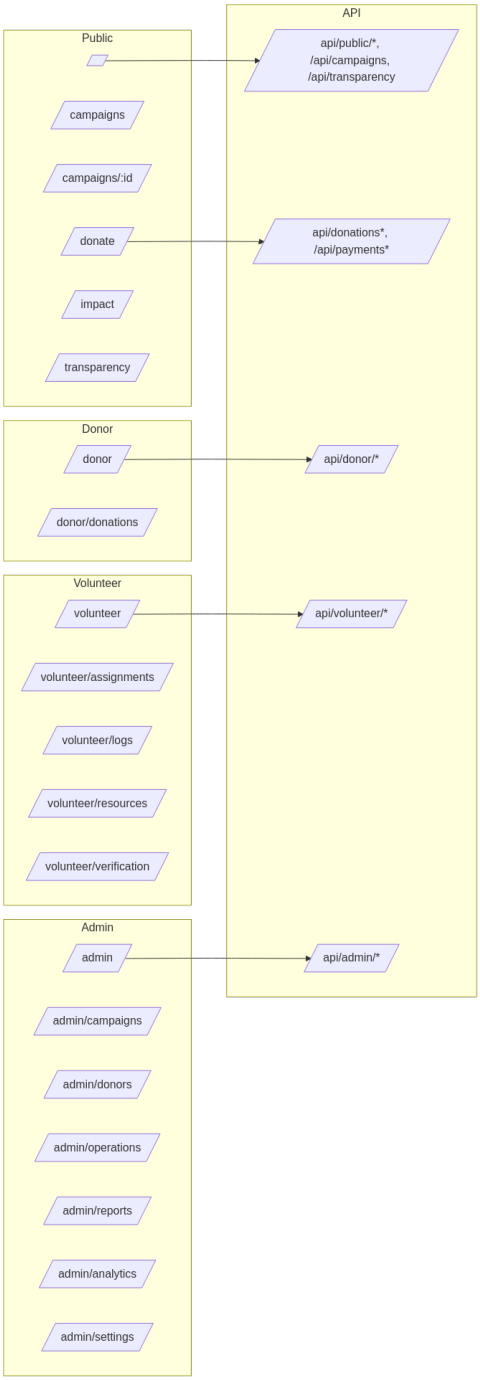

Figure 3: Figure 5.3: Route and module interaction view from current app routes and APIs.

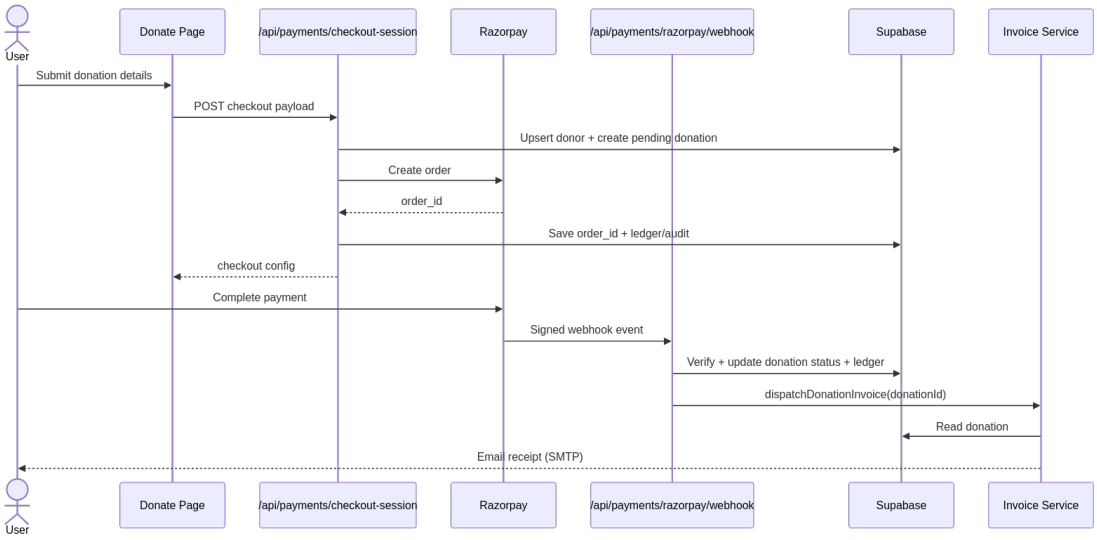

Figure 4: Figure 5.4: Sequence Diagram — Donation Processing Flow.

11.1 Figure 5.5: UI Wireframe Notes

The template includes UI wireframe references for dashboard and project/campaign creation screens. In this revised version, these placeholders are retained as appendix screenshot targets and should be filled using current deployed UI captures from the implemented application routes.

12 CHAPTER 6 METHODOLOGY

The development of ImpactLedger followed an iterative methodology with repeated implementation and validation cycles. Each cycle involved planning, module development, integration checks, and review of observed behavior in API and UI surfaces. This approach was suitable for a community-engaged project where requirements were refined progressively through partner interaction and code-level verification.

The technology stack was selected based on maintainability, ecosystem maturity, and fit to project requirements. The frontend was implemented using React with Next.js. The backend behavior is implemented through Next.js route handlers and server modules. Data persistence uses Supabase PostgreSQL with tenant-scoped schema definitions and reporting views. Payments are integrated through Razorpay APIs and signed webhook processing.

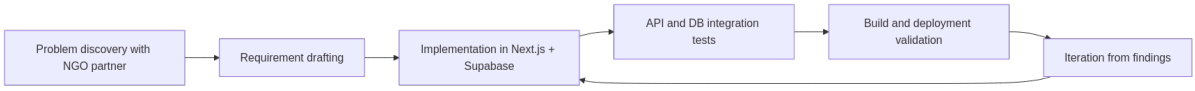

Figure 5: Figure 6.1: Development methodology workflow used for implementation and verification.

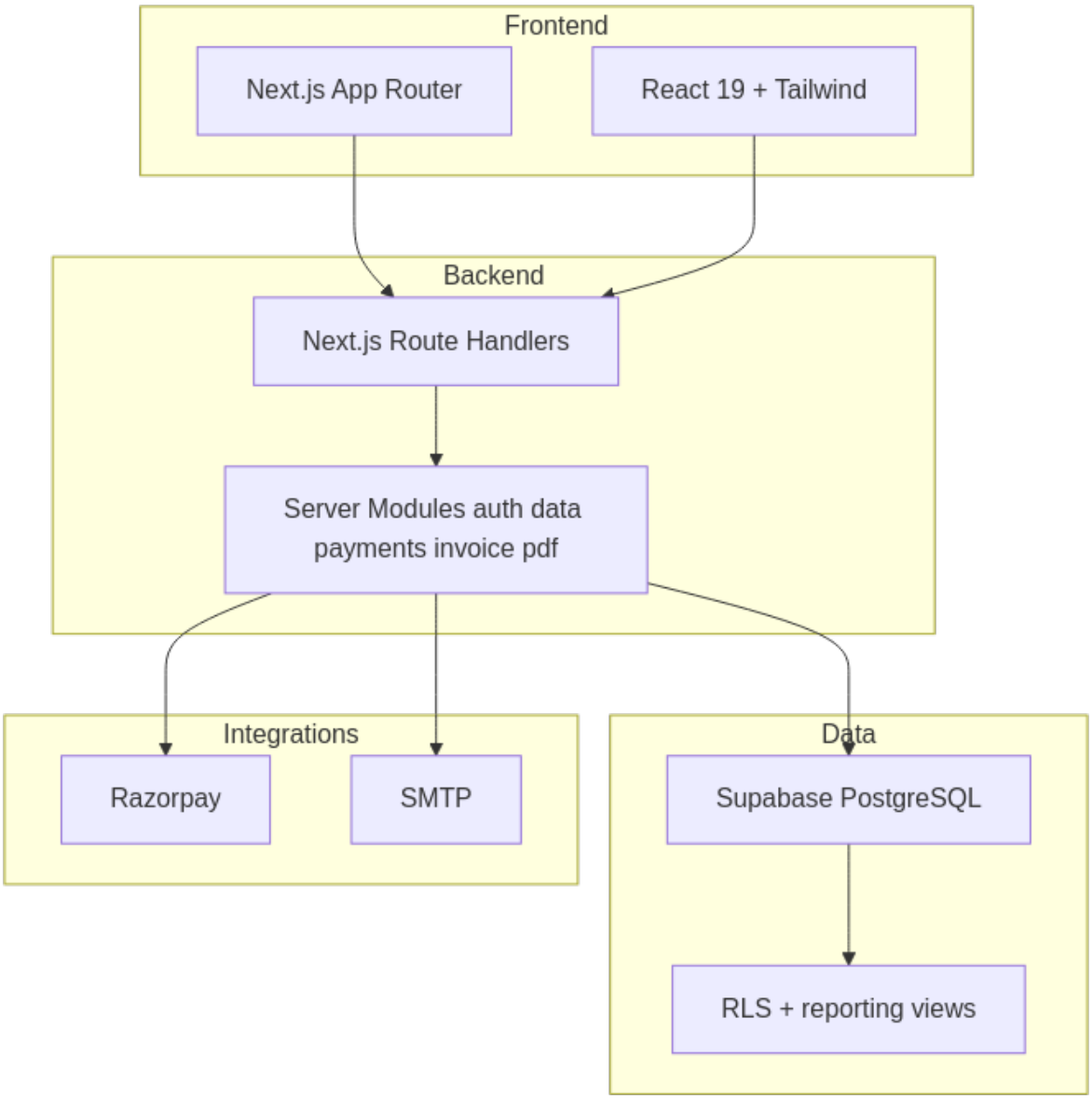

Figure 6: Figure 6.2: Technology stack overview from repository dependencies and modules.

13 CHAPTER 7 IMPLEMENTATION

The implementation of ImpactLedger is organized into three primary modules: frontend, backend, and database integration.

The frontend module, developed using React and Next.js, includes login and signup interfaces, donor/volunteer/admin dashboards, campaign browsing and detail pages, donation pages, transparency views, and role-specific management screens.

The backend module is implemented through Next.js route handlers. It exposes API endpoints for authentication support, campaign retrieval, donation lifecycle handling, payment verification, webhook processing, reporting, analytics, and role-specific dashboards.

The database module uses Supabase PostgreSQL schema objects and views. Relationships are represented through foreign keys and tenant-scoped constraints. Event persistence is handled through ledger and webhook event tables to support auditing.

The deployed application is accessible at https://impact-ledger-cep.vercel.app/ and source code is available at https://github.com/SaptanshuWanjari/ImpactLedger.

Implementation mapping:

- Frontend pages under `src/app/**/page.tsx`
- Backend route handlers under `src/app/api/**/route.ts`
- Server modules under `src/lib/server/*`
- Supabase schema and migrations under `supabase/`

Razorpay checkout, webhook verification, invoice dispatch, and ledger persistence are implemented and connected through these modules.

13.1 Table 7.1: Screenshot Placement Guide for Chapter 7

Figure No.	Screen / View	Capture Instructions	Placement in Report
7.1	Homepage / Landing Page	Capture above-the-fold view of the deployed landing page.	After chapter introduction
7.2	User Registration / Login	Capture login and signup forms with key fields visible.	After authentication module description
7.3	Admin Dashboard	Capture summary cards and at least one chart.	After dashboard module description
7.4	Campaign Creation Form	Capture form with title/category/date/description fields.	After campaign management description
7.5	Project/Campaign Listing	Capture list view with status indicators.	After listing functionality description
7.6	Activity Log Interface	Capture volunteer/admin logging interface.	After activity logging description
7.7	Donation Recording Page	Capture donation entry or donation summary view.	After donation recording description
7.8	Impact Visualization	Capture charts with labeled axes.	After impact dashboard description

14 CHAPTER 8 RESULTS AND IMPACT

The ImpactLedger platform demonstrates working behavior across core functional areas in repository-backed validation paths. API routes for public, donor, volunteer, and admin surfaces are implemented and exercised in integration tests. The frontend renders role-specific and public pages, and data records can be created and retrieved through server handlers.

Code-backed validation signals:

- build passes using `npm run build`;
- integration tests pass (11/11) in `tests/api.integration.test.mjs`;
- payment event lifecycle is captured through `donation_ledger` and `payment_webhook_events`;
- transparency and overview endpoints provide public reporting views.

14.1 Table 8.1: Before vs After Comparison — ImpactLedger Implementation

Parameter	Before digitization (community workflow)	With ImpactLedger implementation
Record keeping	Fragmented manual records and spreadsheets	Structured tenant-scoped database records
Donation tracking	Limited event traceability	Checkout + webhook + ledger event chain
Role visibility	Ad-hoc information sharing	Role-based dashboard and API access
Transparency	Delayed manual reports	Public overview and transparency ledger endpoints
Operational reporting	Manual consolidation effort	Query-backed dashboards and reporting APIs

14.2 Table 8.2: System Performance Metrics — ImpactLedger

Metric	Target	Observed Result	Status
Build integrity	Successful production build	`npm run build` passed	Pass
API integration coverage	Core route validation	11/11 integration tests passed	Pass
Payment reconciliation path	Signed webhook handling	Implemented and persisted	Pass
Audit trail persistence	Event-level storage	Donation/webhook/audit tables present	Pass

15 CHAPTER 9 DISCUSSION

The revised report preserves template structure while replacing outdated technical assumptions with code-backed descriptions. On the technical side, integration across dynamic dashboards, role-specific APIs, and payment reconciliation required careful server-module coordination. Ensuring consistent behavior across donor, volunteer, and admin paths required robust tenant and role handling.

The system has limitations in the current implementation phase. Runtime behavior depends on external services and correct environment configuration (Supabase, Razorpay, SMTP). Some repository validation tooling currently has baseline issues (for example ESLint configuration errors) that are separate from application runtime correctness.

From a project management perspective, aligning documentation with actual implemented behavior required careful claim-by-claim verification against source code and schema artifacts.

16 CHAPTER 10 CONCLUSION AND FUTURE SCOPE

ImpactLedger currently provides a full-stack implementation for NGO campaign transparency, role-based operations, and donation reconciliation. The revised report now aligns technical claims with what is actually implemented in repository code, schema, APIs, and build/test evidence.

Future scope can include richer notification channels, enhanced analytics, deeper automation for reporting exports, and broader mobile-first optimization, provided these are implemented and verified before being described as completed features.

17 QUICK REFERENCE: ALL FIGURES AND TABLES

Figure / Table No.	Chapter	Description
Table 4.1	Chapter 4	Functional Requirements of ImpactLedger
Table 4.2	Chapter 4	Non-Functional Requirements of ImpactLedger
Table 4.3	Chapter 4	Hardware and Software Requirements
Figure 5.1	Chapter 5	System Architecture Diagram
Figure 5.2	Chapter 5	Entity-Relationship Diagram
Figure 5.3	Chapter 5	Route and Module Interaction View
Figure 5.4	Chapter 5	Sequence Diagram — Donation Processing Flow
Figure 6.1	Chapter 6	Development Methodology Workflow
Figure 6.2	Chapter 6	Technology Stack Overview
Table 7.1	Chapter 7	Screenshot Placement Guide
Table 8.1	Chapter 8	Before vs After Comparison
Table 8.2	Chapter 8	System Performance Metrics

18 REFERENCES

GitHub Repository: https://github.com/SaptanshuWanjari/ImpactLedger
Deployed Application: https://impact-ledger-cep.vercel.app/
Next.js Documentation: https://nextjs.org/docs
Supabase Documentation: https://supabase.com/docs
Razorpay API Documentation: https://razorpay.com/docs/api/
React Documentation: https://react.dev/

19 APPENDICES

Appendix A: Screenshots of the ImpactLedger web application (to be inserted as per Table 7.1).